

CS & IT ENGINEERING



Compiler Design

Lexical Analysis

(4-6) Marks {GATE}

TOC + Compiler = 15 marks

Principles, Techniques,
& Tools

Lecture No.- 01



By- Venkat sir

Topics to be Covered



Topic

① 10%

Lexical Analysis

Topic

②

Parsing

50%

Topic

③

Syntax Directed Translation

20%

Topic

④

Intermediate Code Generation

20%

Topic

⑤

Code Optimisation(Dataflow Analysis)

Topic

⑥

Runtime Environment

Grammar

Topics to be Covered



Topic

?????



→ Phases of Compiler

→ Lexical Analysis Phase



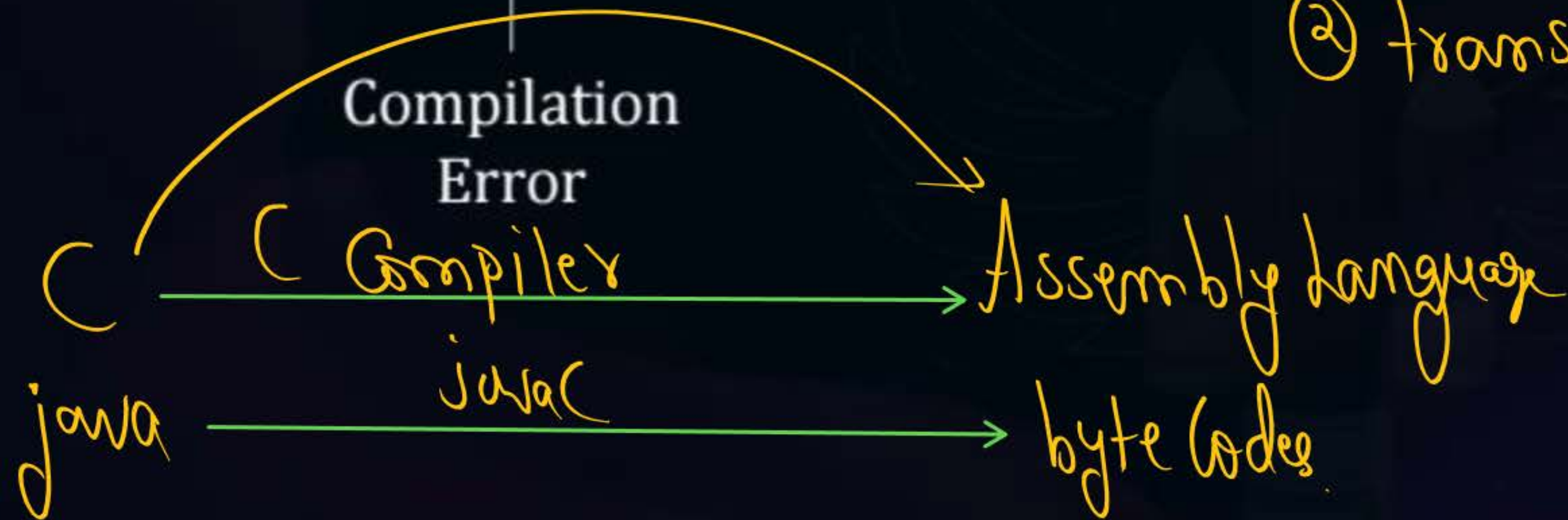
Topic : Introduction of Compiler Design

- Compiler is a translator which converts a program written in one language (Source Language) to other language (Object/Target Language) *and also detects errors.*

FORTRAN
20 years



- ① Error detection
- ② translation

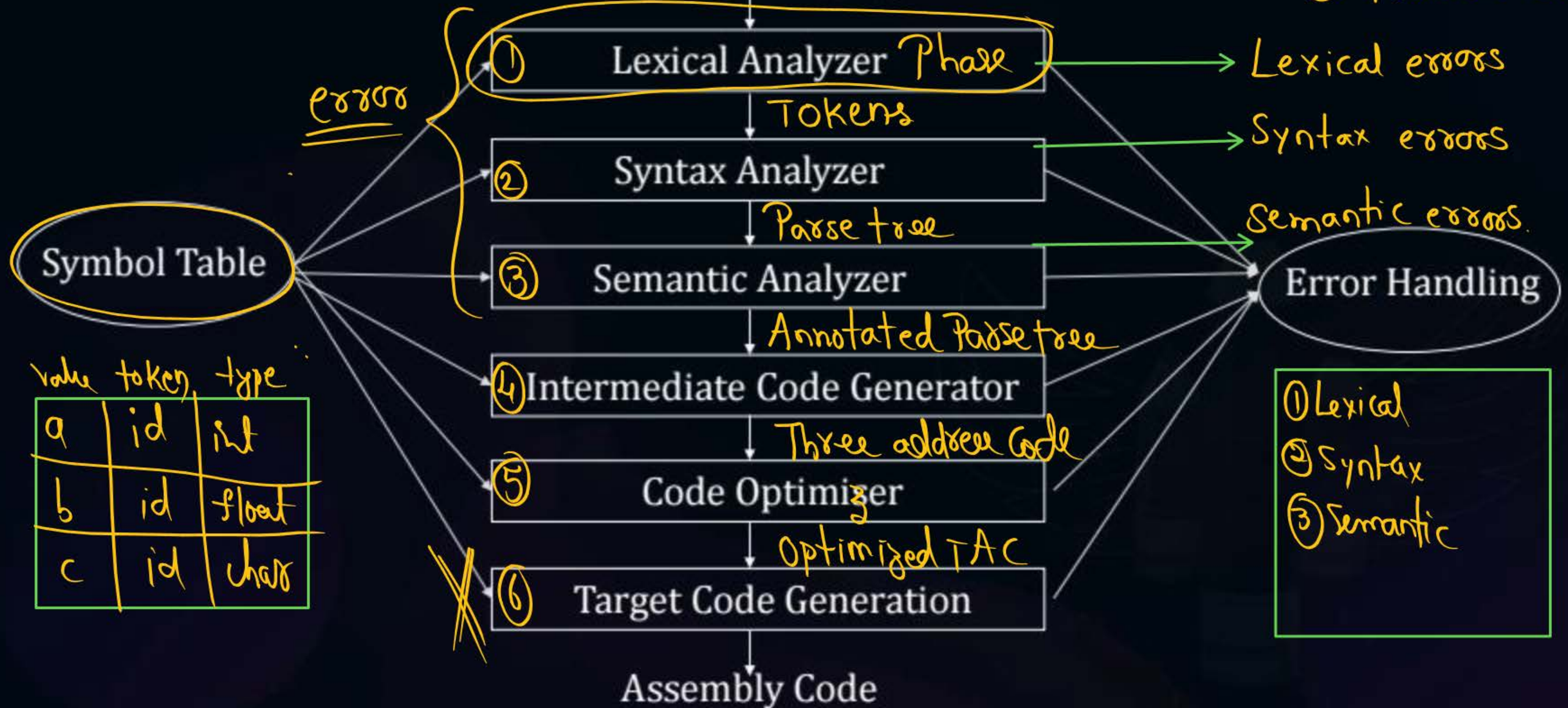




Topic : Introduction of Compiler Design

C, C++,
High Level Language

- ① Error
- ② translation





Topic : Lexical Analysis Phase

- It is a program that takes high level language as input and produces tokens as output.
- It also detects lexical errors present in the program.



Topic : Syntax Analysis Phase

- It is a program that takes **tokens** as input and produces **parse tree** as output.
- It detects **syntax errors** present in the program.



Topic : Semantic Analysis Phase

- It is a program that takes parse tree as input and produces annotated parse tree as output.
- It also detects semantic errors present in the program.(type checking)



Topic : Intermediate Code Generation Phase

- It is a program that translates high level code into intermediate code.
- Advantage of generating intermediate code is to perform optimization in simple way.

Ex:- ① Three address Code

② Syntax tree

③ Directed Acyclic Graph



Topic : Code Optimization Phase

- It is a program that reduces time and space required by the target machine by removing some unnecessary code .
- There are two types of optimizations performed by compilers known as machine independent optimization and machine dependent optimization.



Topic : Code Generation Phase

- It is a program that translates optimized intermediate code into assembly language(or) target code.



Topic : Symbol Table

Symbol Table:

- It is data structure that contains information about identifiers and constants present in the program.

1	int	a
2	float	b
3	char	c



Topic

Error Handler:

- If any phase of compiler detects error then it is stored in error handler.



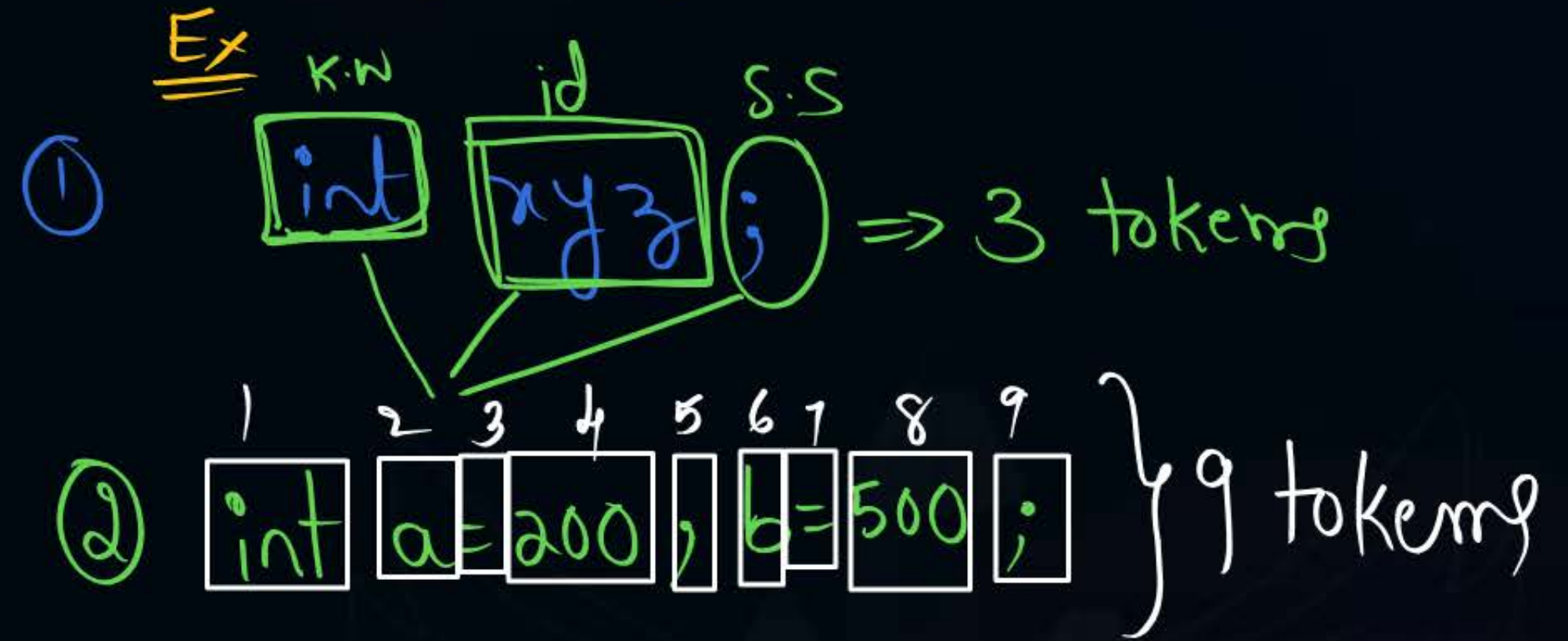
Topic : The Role of Lexical Analyzer

Word

➤ **Token** Token is a group of characters from source code that can be treated as a single logical entity.

➤ Typical tokens are,

- ① Identifiers ✓
- ② keywords ✓
- ③ operators ✓
- ④ special symbols ✓
- ⑤ constants ✓



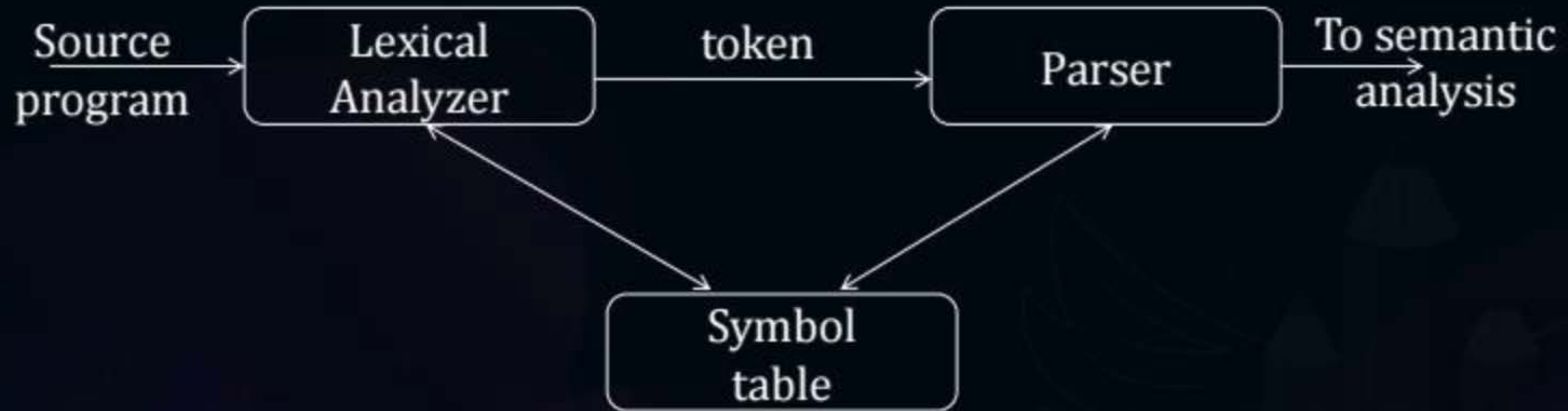


Topic : Functionality of Lexical Analysis

1. It reads source program as stream of characters
2. It produces tokens as output
3. It detects lexical errors.
4. It eliminates comment lines
5. It eliminates white space characters
6. It constructs symbol table



Topic : The Role of Lexical Analyzer





Topic : The Role of Lexical Analyzer

Why to separate Lexical analysis and parsing

- Simplicity of design
- Improving compiler efficiency
- Enhancing compiler portability



Topic : The Role of Lexical Analyzer

- **Token:** Token is a sequence of characters that can be treated as a single logical entity.
- Typical tokens are,
 - 1. Identifiers
 - 2. keywords
 - 3. operators
 - 4. special symbols
 - 5. constants



Topic : Compiler Design

tokens

- ① ——— } Keywords (int, float, for)
- ② ——— } Identifiers (xyz, ab)
- ③ ——— ✓ Constants (10, 2, 0 30)
- ④ ——— ✓ Operators (+, *)
- ⑤ ——— Special symbols ((, .) { } ;)



Topic : Compiler Design

1. Regular expression
2. Finite automat



Topic : Compiler Design

1. `int a = 20;`

Keywords

`int`

`float`

`char`

`for`

`if`

`goto`

`do`

`else`

`goto`

`while`

`short`

`void`



Topic : Compiler Design



$$\underline{\underline{RE}} \quad \underline{\underline{l(l+d)^*}}$$

Identifier

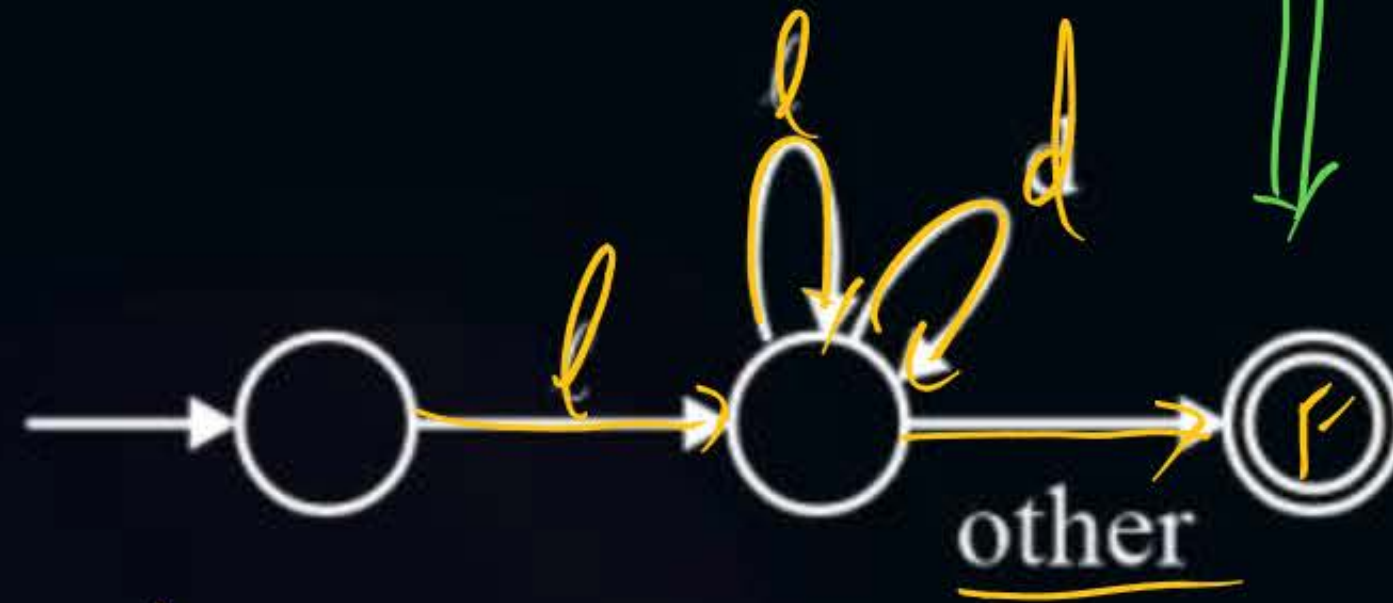
1. Identifiers can be formed using letter and digit.
2. Identifiers can start with letter only
3. Keyword name is not allowed as identifier
4. Identifier can't include any special characters

(exclude -)



Topic : Compiler Design

1. int abc 123



2. int 1 abc xyz 1 2 3

int abcx yz 1 23

3. int a = 5.85;

char
float
int

① xyz123; } 2

① K.W. id.
int xyz;

② int char float int } 4

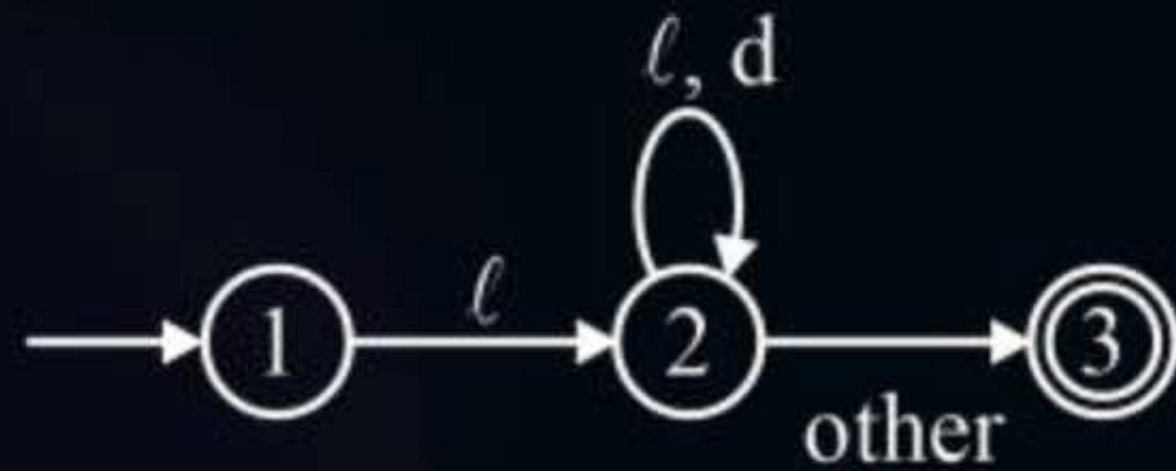


Topic : Compiler Design

Identifier

ID

else
If
Float
int





Topic : Transition diagrams

- Transition diagram for reserved words and identifiers

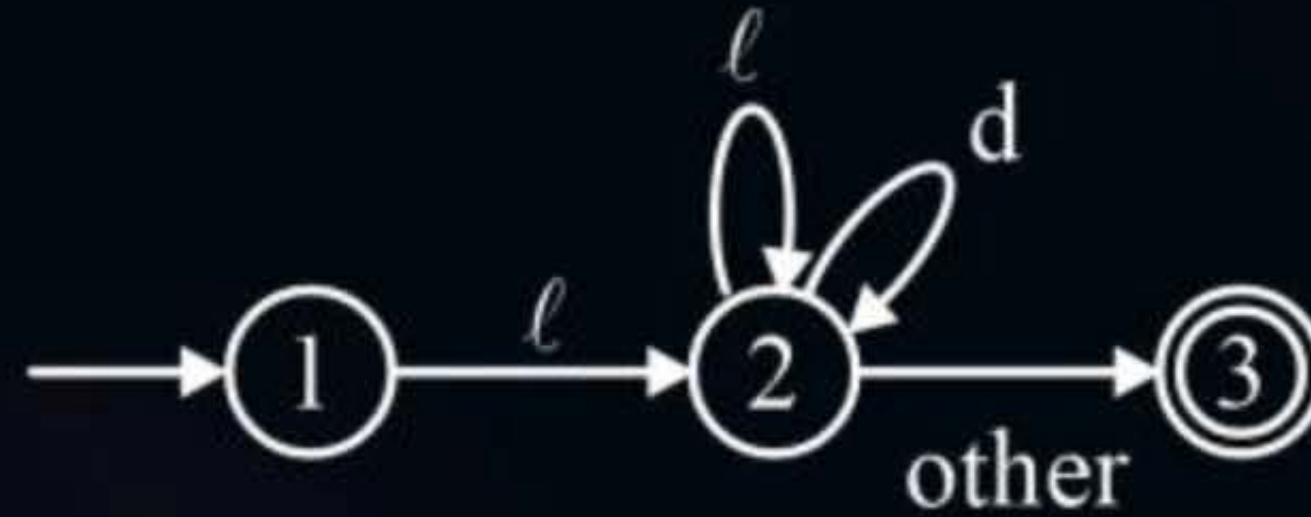




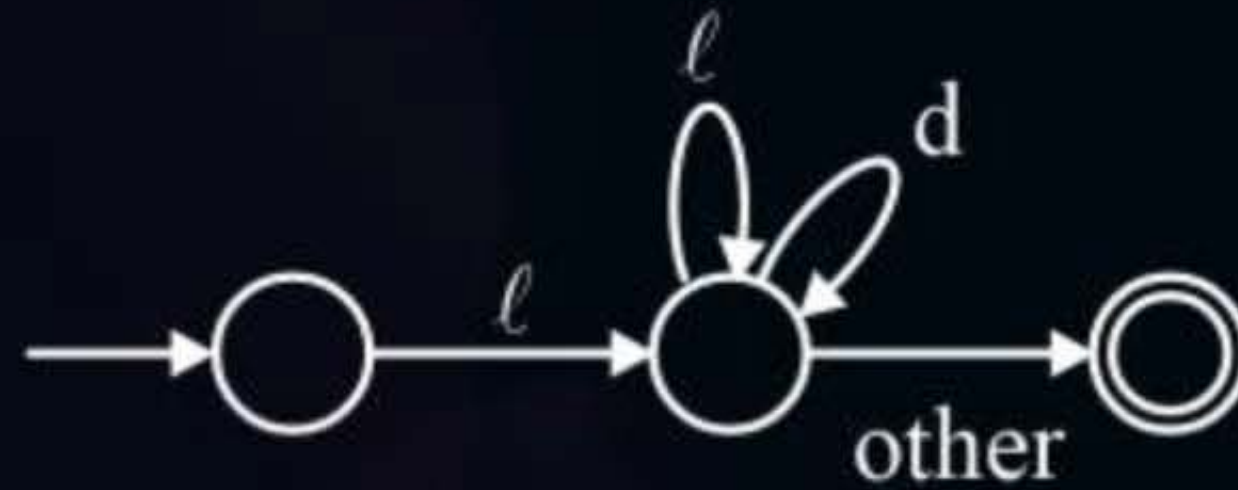
Topic : Compiler Design



$l (l + d)^* \text{ other}$



abc xyz 1 23



intfloartchar;



Topic : Lexical Analyzer

1. int abc ;

2. int a, b, ;

3. abc = b + c ; } 6 tokens

4. if (a < b)

5. int a = 100, b = 200;

6. int abc1234float;

7. char b = 'a'

8. char a = "abc";

9. printf("Compiler Design")



Constants

- 1
a

1

② `printf` (`'Compiler'`); } 5

③ char b = 'a'; } 5 tokens



Topic : Compiler Design

1. `Int a = "compiler";`
2. `int a = "compiler" }`
3. `int a = 20 + "gate" ;`
4. `Printf ("%d%d", x, y) ;`



Topic : Compiler Design

```
main()
```

```
{
```

```
    int a = 10;
```

```
    int b;
```

```
    char c = "abc"
```

```
    b = a + c;
```

```
}
```



Topic : Operator



	Operator
increment/decrement	<code>++</code> , <code>--</code>
Arithmetic operator	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>
relational operator	<code><</code> <code><=</code> <code>></code> <code>>=</code> <code>==</code> <code>!=</code>
Assignment operator	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code>
Bitwise operator	<code><<</code> <code>>></code>



Topic : Compiler Design

- ① $a = b;$ } 4
- ② $a == b;$ } 4 tokens
- ③ $a + b$ } 3 tokens
- ④ $a ++$ } 2 tokens
- ⑤ $a +++ ++ ++$ } 5 tokens
- ⑥ $a < b$ } 3
- ⑦ $a < = b$ } 3
- ⑧ $a += b$ } 3 tokens
- ⑨ $a -= b$ } 3
10. $a > = b$ } 3

⑪ $a = + b$ } 4 tokens



Topic : Compiler Design

```
for (i = 0; i <= 100; i ++)
```

[NAT]



#Q. Consider the following program segment:

```
main ()  
{  
    int a, b, c;  
        a = 50;  
        b = &a;  
    printf("%d", b);  
}
```

The number of tokens in the above C code are ____.

Home Work

[MCQ]

#Q.

If (z == a) function1 (10);

Handwritten annotations: The code is tokenized with green vertical lines and numbers 1 through 11 above them. The tokens are: 1. '(', 2. 'z', 3. '=', 4. 'a', 5. ')', 6. ',', 7. 'function1', 8. '(', 9. '1', 10. '0', 11. ';'.

The number of tokens in the above statement are

A

9

B

11

C

10

D

12



2 mins Summary



Topic

One

Topic

Two

Topic

Three

Topic

Four

Topic

Five

THANK - YOU